

QuotientFlow: Learning Residual Flow Advantage for Symmetry-Reduced CGRA Mapping

HPCA 2027 Submission #4950

Confidential Draft

Do NOT Distribute!!

Abstract—Mapping a dataflow graph (DFG) onto a coarse-grained reconfigurable array (CGRA) requires coupled placement and routing decisions under tight, time-expanded resource constraints. Existing heuristic mappers can make locally short routing choices that consume globally scarce resources, while exact formulations are too expensive to invoke at every search step. This paper presents QUOTIENTFLOW, a deterministic beam mapper that derives state-conditioned guidance from a regularized fractional residual mapping problem. Routing- and compute-capacity duals provide a local first-order approximation to an action’s opportunity cost; a two-tower graph neural network predicts the finite-action residual relative to an exact one-step child relaxation. The resulting FLOWADVANTAGE ranks only legal native placement-and-route actions, and an optional top- k mode applies exact child relaxations to a learned shortlist. A conservative quotient path is defined only for transforms that preserve the complete operation order and induce equivariant legal actions, scores, and successors. We integrate the method with Morpher and two independent legality paths. The current artifact validates integration with 148 regression tests and rejection of all eight deliberately corrupted native mappings. In a two-architecture `txmm` suffix-completion probe after a deterministic 40% length prefix, proposal-only search matched the length baseline’s route cost but required $3.3\text{--}4.0\times$ longer; dual-linear and top-4 variants reached the fixed timeout. These results establish narrow functional correctness and expose relaxation cost. They do not establish end-to-end mapping quality, dual stability, learned-ranking quality, transfer, or quotient reduction.

I. INTRODUCTION

Coarse-grained reconfigurable arrays (CGRAs) combine spatially distributed functional units with a programmable interconnect, providing a flexible substrate for accelerating loop kernels. Their efficiency depends on the compiler’s ability to map a dataflow graph (DFG) onto processing elements and route every dependence through a modulo-expanded interconnect. A poor placement can make later routes impossible even when each early choice appears locally inexpensive. Consequently, mapping remains a central obstacle to retargetable CGRA compilation [1]–[4].

The difficulty is not merely the size of the search space. Placement and routing consume shared resources with highly nonuniform future value. A link on many remaining source-to-sink paths can be far more consequential than an equally long peripheral link. Local distance heuristics do not represent this opportunity cost. Negotiated-congestion routers such as PathFinder recover from overuse iteratively [5], and simulated annealing can revise earlier placements, but both may spend substantial time discovering scarcity indirectly. Exact integer

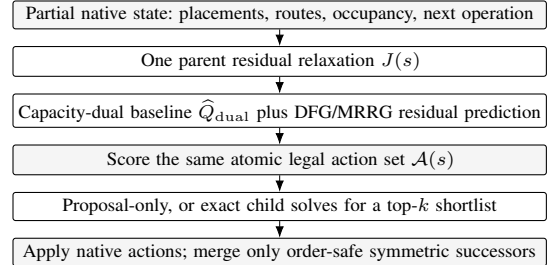


Fig. 1. QUOTIENTFLOW scoring and search contract. Optimization and learning change scores, not the native legal-action universe. Canonicalization is admitted only when the full ordered transition contract is equivariant.

formulations expose global structure but scale poorly when solved repeatedly [7]. Learned mappers can amortize search, but handcrafted labels or end-to-end rewards provide only indirect supervision for the resource bottleneck that determines routability [8], [9].

Our key observation is that a partial mapping state defines a residual optimization problem whose routing-capacity dual variables approximate local marginal scarcity. Strong convexity makes the primal optimum unique, but it does not by itself make capacity multipliers unique under redundant constraints. We therefore treat the extracted shadow prices as a solver- and regularization-dependent first-order baseline whose stability must be measured. A discrete action can alter feasible endpoint assignments, path alternatives, and future cuts in ways that a parent-state linearization cannot capture, motivating a learned residual correction rather than an unconstrained policy.

QUOTIENTFLOW combines two candidate reductions. First, FLOWADVANTAGE ranks legal placement-and-route actions by adding a learned residual advantage to the parent relaxation’s dual linearization; optional top- k correction solves exact child relaxations only for the learned shortlist. Second, a conservative canonicalization can merge partial states only when an admitted transform is a symmetry of the complete ordered transition system. The first mechanism is intended to reduce child solves; the second is intended to remove duplicate state orbits. Neither changes the native candidate generator, but both require empirical validation before a quality or search-reduction claim is warranted.

Figure 1 summarizes the scoring contract. We implement the method over Morpher’s native DFG and modulo routing resource graph (MRRG) contracts [10]. The reference mapping

is used only to identify a validated corpus; its placements, routes, occupancy, and score are not inputs. The available learned-method probe begins from an empty mapper state but delegates the first 40% of operations to a deterministic length scorer. It is therefore a suffix-completion experiment, not end-to-end evidence for learned early decisions. Complete mappings are checked by both an independent serialized-contract validator and a native reimport path.

This paper makes four contributions:

- A regularized fractional residual placement-and-routing formulation whose capacity multipliers define a testable first-order action baseline.
- FLOWADVANTAGE, a residual-learning objective and two-tower GNN that predicts finite-action corrections while preserving the native legal-action universe.
- An order-safe state-orbit specification, deterministic beam search, and optional top- k exact correction, with exactness claims explicitly conditioned on transition and score equivariance.
- A Morpher-native integration and evidence protocol that distinguishes legal mapping failure, timeout, and infrastructure error. The supplied evidence supports narrow correctness and runtime observations, not mapping-quality improvement.

II. BACKGROUND AND MOTIVATION

A. CGRA mapping

A scheduled DFG is a directed graph $G_D = (V_D, E_D)$. Each operation $v \in V_D$ has an opcode and a legal modulo-time interval. For initiation interval (II) I , the architecture is expanded into an MRRG $G_R = (V_R, E_R)$ whose resources represent time-indexed datapaths, ports, switches, and links. A complete mapping assigns every operation to a compatible datapath and every dependence to a directed resource sequence satisfying temporal, occupancy, signal, memory-access, and recurrence constraints.

The mapper operates incrementally. A partial state s contains operation placements P_s , routed dependences R_s , occupied compute resources C_s , and occupied routing resources L_s . A legal action a places the next operation and atomically routes each newly closed dependence. Atomicity is important: evaluating a target placement without its required routes admits states that cannot occur in the native mapper.

The operation order is deterministic: ascending schedule level, descending fan-out, then stable node identifier. For a state s , the candidate generator enumerates compatible datapaths within the operation’s schedule horizon, enumerates up to k_p native routes per newly closed dependence, combines compatible route choices, and rejects actions that violate occupancy or signal compatibility. Search policies therefore compare scores over the same legal action set $\mathcal{A}(s)$.

Three invariants constrain the design. *Native legality* means that scoring never creates a placement or route outside the native candidate generator. *Atomic action semantics* means that a placement is evaluated together with every dependence

TABLE I
NOTATION USED THROUGHOUT THE PAPER. A BAR DENOTES RESIDUAL CAPACITY AFTER APPLYING THE NATIVE PARTIAL STATE.

Symbol	Meaning
$G_D = (V_D, E_D)$	Scheduled dataflow graph
$G_R = (V_R, E_R)$	II-expanded routing resource graph
$s; s_a$	Partial native state; child after action a
$\mathcal{A}(s)$	Atomic legal actions exposed at s
$J(s)$	Optimal residual fractional objective
λ_ℓ, μ_r	Routing- and compute-capacity duals
$Q^*(s, a)$	Exact one-step relaxed action value
$A_{\text{res}}(s, a)$	Residual error of the dual estimate
$\kappa(s)$	Exact canonical state key

that becomes routable at that step. *Score-only comparability* means that deterministic beam variants differ in scoring, not in candidate caps, route enumeration, operation order, or terminal checks. These invariants are required for attributing an outcome to guidance rather than to a different search universe.

B. Why path length is insufficient

A length-based policy minimizes the resources used by the current action. Let $c(a)$ be the sum of base costs for newly occupied routing resources plus any immediate placement cost. Two actions with the same $c(a)$ can have different consequences. One may consume a bridge in the residual graph, remove the only path between a remaining producer and consumer, or occupy a datapath needed by an unmatched opcode. These effects are state dependent and cannot be represented by a static edge weight.

A direct exact lookahead would apply each $a \in \mathcal{A}(s)$, solve the residual mapping problem for child state s_a , and rank by $c(a) + J(s_a)$, where J is the relaxed residual objective. This is principled but expensive: a beam state may expose hundreds or thousands of native actions, and each child problem includes many flow variables. The design goal is therefore to approximate the ordering induced by exact child relaxation while solving at most one parent relaxation per state and a small number of child relaxations.

C. Dual prices as local sensitivity

For a convex residual problem, a capacity multiplier participates in a first-order sensitivity relation for marginal right-hand-side changes under standard regularity conditions [11]. A positive routing multiplier can therefore indicate that residual demand presses against that resource. However, the multiplier selected by a numerical solver need not be unique when constraints are redundant, and a native action removes a finite unit rather than an infinitesimal amount. We use the signal as an approximation derived from complete residual demand, not as a measured or solver-independent scarcity quantity.

Dual sensitivity is still a local approximation. Discrete actions remove whole units of capacity, change endpoint feasibility, and can alter the active set. The parent dual may thus rank many actions correctly while missing state transitions near cuts or compatibility boundaries. QUOTIENTFLOW uses

the dual estimate as a structured baseline and learns only the nonlinear residual.

III. RESIDUAL MAPPING FORMULATION

A. Variables and residual capacities

Let $U_s = V_D \setminus \text{dom}(P_s)$ be the unmapped operations and $D_s \subseteq E_D$ the unrouted dependences. For each $v \in U_s$ and compatible compute resource $r \in V_R$, $x_{v,r} \in [0, 1]$ denotes fractional placement. For each residual dependence $e \in D_s$ and directed MRRG edge $\ell \in E_R$, $f_{e,\ell} \in [0, 1]$ denotes fractional flow. Existing placements appear as fixed one-hot endpoint vectors. Existing routes and any optional knockout resource have zero residual capacity.

For residual dependence $e = (u, v)$ and routing node q , let $B_{q,\ell}$ be the directed incidence matrix, with +1 at the source and -1 at the destination of edge ℓ . Let $\pi_v(q)$ denote either the fixed placement of an anchored operation or the fractional placement vector of an unmapped operation. The residual program is

$$J(s) = \min_{x,f} \sum_{e \in D_s} \sum_{\ell \in E_R} c_\ell f_{e,\ell} + \frac{\tau}{2} \sum_{v,r} x_{v,r}^2 + \frac{\tau}{2} \sum_{e,\ell} f_{e,\ell}^2 \quad (1)$$

$$\text{s.t.} \quad \sum_{r \in \mathcal{C}(v)} x_{v,r} = 1, \quad \forall v \in U_s, \quad (2)$$

$$\sum_{v \in U_s} x_{v,r} \leq \bar{c}_r(s), \quad \forall r, \quad (3)$$

$$\sum_{\ell \in E_R} B_{q,\ell} f_{e,\ell} = \pi_u(q) - \pi_v(q), \quad \forall e \in D_s, \forall q \in V_R, \quad (4)$$

$$\sum_{e \in D_s} f_{e,\ell} \leq \bar{b}_\ell(s), \quad \forall \ell, \quad (5)$$

$$0 \leq x_{v,r} \leq 1, \quad 0 \leq f_{e,\ell} \leq 1.$$

Here $\mathcal{C}(v)$ is the compatible compute-resource set, $\bar{c}_r(s)$ is residual compute capacity, and $\bar{b}_\ell(s)$ is residual routing capacity. The base cost c_ℓ distinguishes movement and register/wait resources. The quadratic term with $\tau > 0$ makes the primal optimum unique. It is intended to reduce numerical variation, but it does not guarantee unique capacity multipliers; dual-ranking stability must therefore be established by a τ /solver/tolerance sweep. The implementation uses CVXPY [12], with a conic solver as primary and OSQP [13] as fallback.

Let $\lambda_\ell(s) \geq 0$ be the dual of (5), and $\mu_r(s) \geq 0$ the dual of (3). The solver records assignment, flow, and capacity residuals; variable bounds; status; objective; solve time; and the fraction of positive duals. A state is scoreable only when the residual solve returns a permitted terminal status and passes numerical-health thresholds.

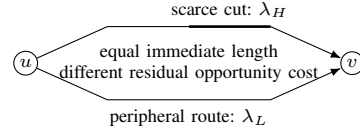


Fig. 2. Conceptual motivation for the dual baseline. Equal-length native routes can consume resources with different local capacity sensitivities. The finite child relaxation is still required to label errors caused by endpoint and active-set changes.

B. Dual-linear action estimate

A legal action a occupies a target compute resource $r(a)$ and a set of previously free routing resources $L(a)$. Its parent-state dual estimate is

$$\hat{Q}_{\text{dual}}(s, a) = c(a) + \mu_{r(a)}(s) + \sum_{\ell \in L(a)} \lambda_\ell(s). \quad (6)$$

This is a first-order approximation of the objective increase caused by removing the action's resources from the parent residual problem. It is deterministic, permutation equivariant with respect to verified resource relabelings, and inexpensive once $J(s)$ has been solved.

Figure 2 illustrates why the baseline can distinguish equal-length routes while remaining only local. The oracle one-step value solves the exact relaxed child:

$$Q^*(s, a) = c(a) + J(s_a) - J(s), \quad (7)$$

where s_a is the legal child state. We define the residual flow advantage

$$A_{\text{res}}(s, a) = Q^*(s, a) - \hat{Q}_{\text{dual}}(s, a). \quad (8)$$

The model predicts A_{res} , not Q^* directly. This decomposition supplies an interpretable baseline and permits dual-only and direct- Q^* ablations. Whether it reduces target difficulty or improves held-out ranking is an empirical question, not a consequence of the definition.

IV. THE QUOTIENTFLOW MAPPER

A. Graph representation and action features

The model has separate towers for the DFG and MRRG. DFG node features encode operation type, normalized in/out degree, schedule phase, anchored status, and normalized anchored coordinates. The DFG tower performs five directed residual message-passing steps with separate incoming and outgoing aggregation. MRRG node features encode normalized space-time coordinates, degree, occupancy, boundary indicators, and a log-normalized compute dual. MRRG edge features encode resource kind, occupancy, residual availability, direction, and a log-normalized routing dual; the tower performs five directed edge-conditioned residual steps.

For action a , the head concatenates the next-operation and target-resource embeddings; mean and max pools of each graph; mean and max embeddings of newly occupied routing edges; and deterministic scalar features. The scalar features include immediate cost, occupied-resource fractions, alternative-path

availability, bridge and edge-betweenness exposure, demand-to-cut ratios, parent objective and residual-health statistics, and the routing and compute terms of (6). Scalar features are standardized using the training split only.

B. Labels and residual-learning objective

Training groups are partial states with a common next operation and their complete bounded legal action set. For every retained action, label generation solves the exact child relaxation and computes (7) and (8). Let m and σ be the median and robust scale of training residuals. The clipped regression target is $y_a = \text{clip}((A_{\text{res}}(s, a) - m)/\sigma, -8, 8)$. At inference,

$$\hat{Q}_{\text{FA}}(s, a) = \hat{Q}_{\text{dual}}(s, a) + \sigma g_{\theta}(s, a) + m. \quad (9)$$

Lower values are preferred.

The groupwise training loss is

$$\mathcal{L} = \mathcal{L}_{\text{list}} + \frac{1}{2}\mathcal{L}_{\text{pair}} + \frac{1}{4}\mathcal{L}_{\text{Huber}}, \quad (10)$$

where $\mathcal{L}_{\text{list}}$ is cross entropy from a soft oracle distribution over exact action regrets, $\mathcal{L}_{\text{pair}}$ is a regret-weighted logistic loss over materially separated pairs, and $\mathcal{L}_{\text{Huber}}$ regresses y_a . Checkpoint selection is lexicographic: top-4 oracle recall, epsilon-optimal top-1, relative selected regret, and exact top-1. This criterion matches the intended use as a proposal mechanism rather than rewarding only global correlation.

C. Optional Exact Top- k Correction

Proposal-only search ranks all legal actions by (9). Top- k search uses the same scores to select k actions and replaces their scores with exact values from (7); all other actions receive infinite local score for that state. The comparison to full lookahead is valid only when both methods enumerate the same legal actions and differ solely in the number of child relaxations. Caches are keyed by architecture, DFG, partial-state, action, solver configuration, and source hashes to prevent cross-contract reuse.

D. Deterministic beam search

Figure 3 gives the complete search. Beam entries are ordered by cumulative score, action key, and stable state key. Successors with identical serialized native state are merged. A scorer may reject one state if its relaxation or feature contract is unavailable; other beam states continue. The mapper stops only on a complete legal mapping, no legal action, expansion limit, cancellation, or external timeout.

E. Order-safe symmetry quotienting

Let $v_1 \prec \dots \prec v_n$ be the complete deterministic operation order, including the stable-identifier tie break, and let $\rho(v_i) = i$. A DFG automorphism is admissible only if it preserves opcode, directed typed edges, schedule semantics, and the total rank ρ . Because ρ is a total order, this condition may reduce the admissible DFG group to the identity. This restriction is deliberate: graph automorphism alone is insufficient when search transitions depend on a fixed ordered prefix.

```

1: Input: native problem  $\mathcal{P}$ , initial frontier  $B_0$ , scorer  $S$ ,
   beam width  $w$ 
2:  $B \leftarrow B_0$ 
3: for operation  $v$  in deterministic order do
4:    $C \leftarrow \emptyset$ 
5:   for each  $(q, s) \in B$  do
6:      $T \leftarrow$  compatible targets for  $v$  in  $s$ 
7:      $A \leftarrow$  atomic legal placement-route actions from  $T$ 
8:      $z \leftarrow S(\mathcal{P}, s, A)$ 
9:     for each finite-scored  $(a, z_a)$  do
10:       $s' \leftarrow \text{Apply}(s, a)$ 
11:      insert or improve  $C[\text{Key}(s')]$  with  $(q + z_a, s')$ 
12:    end for
13:  end for
14:  if  $C = \emptyset$  then
15:    return valid mapping failure
16:  end if
17:   $B \leftarrow$  first  $w$  entries of sorted  $C$ 
18: end for
19: for each complete state in sorted  $B$  do
20:   if independent legality passes then
21:    return mapping
22:   end if
23: end for
24: return legality failure

```

Fig. 3. Deterministic native beam mapping. Candidate generation and legality are identical across scoring policies.

For a state at depth d , an admissible joint transform $\gamma = (\gamma_D, \gamma_R)$ must also preserve the mapped prefix, architecture resource and edge classes, capacities, time phase, operation support, memory and recurrence semantics, and every native conflict relation used by action generation. In addition, it must satisfy the ordered transition contract

$$\gamma \mathcal{A}(s) = \mathcal{A}(\gamma s), \quad (11)$$

$$\gamma \text{Apply}(s, a) = \text{Apply}(\gamma s, \gamma a), \quad (12)$$

$$z_s(a) = z_{\gamma s}(\gamma a), \quad (13)$$

where the first equality denotes a bijection of legal atomic actions and z is the active local scorer, including deterministic tie handling. A transform is not admitted merely because it is an automorphism of the unannotated DFG or MRRG.

For the admitted group $\Gamma(s)$, define $\text{ser}(\gamma s)$ from the transformed ordered prefix, placements, compute occupancy, routing occupancy, routed-edge identifiers, and remaining operation sequence. The canonical key is

$$\kappa(s) = \min_{\gamma \in \Gamma(s)} \text{ser}(\gamma s). \quad (14)$$

Search retains the lowest cumulative-score representative per key. Under the three equivariance conditions above and invariant terminal legality and route cost, the orbit relation is a weighted bisimulation: exhaustive quotient and nonquotient searches have the same terminal orbits and best legal cost.

For a finite beam, removing duplicate members of one orbit can expose additional non-equivalent states, so we claim a dominance reduction rather than identical heuristic output. No truncated group is called exact; if full verification or enumeration is unavailable, quotienting is disabled and ordinary byte-identical state merging remains enabled. Exhaustive small-instance action-bijection and terminal-set tests remain required before reporting quotient results.

V. NATIVE MORPHER INTEGRATION

A. Contract extraction

Morpher supplies a scheduled DFG, an II-expanded MRRG, and a native mapping schema [10]. The adapter imports operation keys, opcodes, basic-block mutex relations, datapath ownership, functional-unit support, operation latencies, resource conflicts, and directed MRRG connectivity. Missing latency or compatibility metadata fails closed. The mapper never synthesizes a default operation latency and never treats a structural JSON parse as evidence of functional legality.

A native state records operation placements, datapath occupancy, resource-to-signal occupancy, and routes keyed by DFG dependence. Native action generation reuses the architecture’s route semantics, including broadcasts, mutex compatibility, mux conflicts, temporal deadlines, and memory-access restrictions. The final state exports through the same schema consumed by the independent validator and native reimport path.

B. Reference isolation and suffix-probe initialization

A reference mapping is hash-checked only to establish corpus provenance and native input consistency. It is forbidden as a source of placement, route, occupancy, score, or latency information during a paper run. The available probe initializes the mapper from an empty state, after which a deterministic length scorer maps

$$d_0 = \min\{|V_D| - 1, \max(1, \lceil 0.40|V_D| \rceil)\} \quad (15)$$

operations. The complete beam frontier at depth d_0 is passed to the selected method, which controls only the suffix. Prefix time, expansions, actions, and routing attempts are reported separately. Because the learned method cannot revise the first d_0 decisions, this protocol is not used for a headline end-to-end claim. The frozen headline protocol sets $d_0 = 0$ for deterministic methods and reports 0%, 20%, and 40% sensitivity only as a secondary analysis; conventional baselines start from their own empty native state under the same end-to-end wall budget.

C. Legality and terminal status

The independent checker operates only on serialized DFG, MRRG, and mapping documents. It verifies unique compute occupancy, directed route connectivity, opcode compatibility, schedule bounds, loop-carried timing, memory access, architecture identity, and II identity. A second path reconstructs a fresh native MRRG and imports the mapping. A result is a legal success only if the mapper reports completion and the required legality checks pass.

TABLE II
FIXED MAPPING METHODS. ALL LEARNED AND DETERMINISTIC BEAM METHODS USE THE SAME NATIVE CANDIDATE GENERATOR.

Method	Action score / search mechanism
Length	Immediate native route length
Dual-linear	Eq. (6)
Proposal	Learned score in Eq. (9)
Top- k	Proposal shortlist, $k \in \{1, 2, 4, 8\}$ exact child solves
Full lookahead	Exact child solve for every legal action
PathFinder	Native negotiated-congestion mapper
Simulated annealing	Native mapper, seeds 11, 23, and 37

Terminal status is part of the scientific result. VALIDMAP-PINGFAILURE means the mapper executed normally but exhausted its legal frontier. TIMEOUT means an external wall bound terminated the process tree. ERROR denotes an invalid contract or infrastructure failure. Pending and running rows are nonresults. Aggregate quality comparisons must use identical paired sets of normally completed cells.

VI. EXPERIMENTAL METHODOLOGY

A. Questions

The complete evaluation is organized around five questions:

- 1) Do residual duals identify consequential routing resources?
- 2) Does learned residual correction improve legal mapping success, minimum II, or route cost over length and dual-linear scoring?
- 3) How much exact child-lookahead quality does top- k retain, and at what runtime?
- 4) Does exact quotienting reduce explored states without changing the best result?
- 5) Does the method transfer across architecture topology, memory constraints, and array size?

The supplied handoff contains narrow correctness evidence and a limited native probe, but not the paired or held-out data required to answer Questions 1–5. We report only the available results and define the evidence needed for each future claim.

B. Workloads and architectures

The frozen paper suite uses six established native kernels (array_add, array_cond, gemm_nt, fix_fft, hpcg, and trmm) and four compiler-derived kernels (fir, conv2, conv3, and mac). Kernel selection precedes observation of QUOTIENTFLOW outcomes. The architecture set comprises A0, a standard 4×4 HyCUBE; A1, a standard 4×4 NoC; A2, a memory-constrained 4×4 HyCUBE; and A3, a held-out 8×8 HyCUBE [6]. A3 is excluded from training, checkpoint selection, and threshold selection.

C. Metrics and pairing

For each kernel, architecture, seed, and budget cell, the artifact records status, legality, success, II attempts, minimum mapped II, route cost, mapped operations, expansions, generated actions, routing attempts, parent and child solves, cache hits, peak memory, and stage times. Quality deltas

are computed only on identical paired terminal cells. The planned aggregate uses 10,000 paired bootstrap resamples with fixed seed 24072026 and reports the estimate, 95% interval, paired count, and per-kernel/per-architecture values. An interval including zero is not described as statistically reliable.

D. Evidence provenance and reproducibility

Each run is an isolated process with an immutable manifest, heartbeat, external process-tree timeout, schema-validated temporary result, file-system synchronization, and atomic rename. The record binds source, configuration, checkpoint, kernel, architecture, solver, and native-input hashes. Cache keys include the partial state and solver contract, preventing reuse across semantically different problems. The architecture and kernel census is frozen before reading learned-method outcomes; A3 remains excluded from model and threshold selection.

The evaluation treats operational state as data. A legal-success row requires method completion and the configured legality checks. A normal search exhaustion is a valid mapping failure. Timeout, error, unsupported, running, and pending remain distinct statuses and are never inserted into a success denominator without an explicitly named success-under-budget metric. This distinction is essential when solver overhead differs sharply between methods.

E. Learning and dual-validation disclosures

Any quality-bearing learned result must report the split unit and complete train/validation/test membership; counts of state groups and actions; trajectory-level leakage prevention; model dimensions and parameter count; optimizer, learning-rate schedule, batch construction, epochs, seeds, checkpoint rule, and training cost; and exact-child label success, rejection, and fallback rates. Held-out reporting must include top-1 and top- k oracle recall, selected regret, and rank correlation for dual-only, scalar-only, GNN-only, no-dual, direct- Q^* , and residual-target variants. The current handoff does not contain these values, so the paper does not claim reproducible learned ranking.

Dual validation is separate from model validation. On held-out states, the frozen analysis compares each positive multiplier with a finite resource-knockout difference, evaluates action-regret rank correlation and top- k recall, sweeps τ , solver, and tolerances, and reports positive-dual, fallback, rejected-state, and solver-disagreement rates. Categorical wording such as “quantifies scarcity” is not used unless these measurements support it.

VII. RESULTS

A. Functional correctness

Table IV summarizes the evidence available in the handoff. The repository’s regression suite reports 148 passing tests with plugin autoload disabled to avoid an unrelated environment dependency. These tests cover architecture and DFG construction, flow conservation, relaxation health, equivariance, native mapper state transitions, proposal compatibility, caching, seeded simulated annealing, manifest construction, memory layout, and queue semantics.

TABLE III
EVIDENCE AVAILABLE AT ARTIFACT FREEZE AND ITS ADMISSIBLE INTERPRETATION.

Evidence	Supports
Regression and adversarial legality tests	Native-state and checker correctness
Two-architecture terminal proposal probe	Executability, legality, and measured overhead
Incomplete length snapshot	Protocol coverage; no paired method delta
Top-4 timeout rows	Runtime boundary; no mapping failure
Missing paired core matrix	No success, II, route-cost, or portability claim

TABLE IV
AVAILABLE CORRECTNESS EVIDENCE. “NATIVE” DENOTES A FRESH NATIVE RECONSTRUCTION/REIMPORT CHECK.

Check	Passed	Total
Regression tests	148	148
Independent negative legality	8	8
Native negative legality	8	8
Deterministic prefix repeatability	2	2
Atomic native PathFinder smoke	1	1

The stronger negative test begins from a legal native `gemm_nt` mapping and independently introduces eight faults. They cover duplicate compute occupancy, disconnected direction, incompatible multiplication placement, schedule-window violation, recurrence overrun, illegal memory placement, wrong architecture hash, and wrong II. Both the serialized-contract checker and the native reimport validator reject all eight through structured validation paths; crashes and timeouts are not credited as rejection.

B. Baseline coverage

The extended length-baseline snapshot contains 13 immutable rows. Eight are normally terminal: three legal successes and five valid mapping failures. One row timed out, one reported a contract error, two were still running, and one was pending at artifact freeze. Table V reports the rows without imputing outcomes. The three successful `trmm` cells map all 11 operations at II 4 with route costs 71, 72, and 71 on A0, A1, and A2, respectively. The valid mapping failures stop at 15/20 operations for `array_add` A0, 20/25 and 16/25 for `array_cond` A1/A2, and 12/24 and 15/24 for `hpcg` A0/A1.

This snapshot is insufficient for a comparative claim because the learned and conventional baselines do not share the same complete paired terminal set. It documents native baseline coverage and the cost of exact route enumeration, particularly on A3; it does not validate the 40% suffix protocol as an end-to-end comparison.

C. Strict native FlowAdvantage probe

Table VI presents the only complete cross-method probe in the supplied artifact. It uses `trmm` on A0 and A2, a

TABLE V
LENGTH-BASELINE SNAPSHOT. NONTERMINAL AND ABNORMAL ROWS ARE
COVERAGE INFORMATION, NOT QUALITY OUTCOMES.

Status	Rows
Legal success (DONE)	3
VALIDMAPPINGFAILURE	5
TIMEOUT	1
ERROR	1
Running	2
Pending	1
Total	13

deterministic 40% length prefix, beam width 4, action limit 24, at most 5000 expansions, and a 600-s external wall bound. Length and proposal-only search complete legal mappings on both architectures with identical route cost 71. Proposal-only performs 22 parent relaxations and no child solves. Its wall time is 468.48 s on A0 and 481.49 s on A2, compared with 117.44 s and 146.61 s for length, corresponding to $3.99\times$ and $3.28\times$ overhead.

Dual-linear reaches six of eleven operations after four parent solves on each architecture and times out. Top-4 reaches five operations and times out before a complete result. A timeout is not evidence of mapping failure, so neither method contributes a route-cost or success comparison. The probe demonstrates that model-guided native scoring is executable and legal, while repeated parent/child relaxation dominates the present implementation.

D. What the current results establish

The evidence supports three conclusions. First, the native adapter and state transitions are testable under both positive and adversarial legality checks. Second, the proposal scorer can complete a native mapping without consuming the reference witness and without changing the legal action universe. Third, relaxation cost is currently too high for the intended top- k hierarchy under a 600-s cell budget. The evidence does *not* establish improved mapping success, II, route cost, portability, or quotient-search reduction. Those claims require the paired matrix and ablations described in Section VI.

VIII. DISCUSSION

A. What remains to validate about the first-order baseline

The probe establishes neither usefulness nor stability of the dual decomposition. It does identify an implementation constraint: solving the native residual program at each beam state must be amortized or compiled more aggressively. The parent multiplier score is attractive because it is derived from complete residual demand, yields a transparent dual-linear ablation, and can be compared directly with finite-resource knockout differences. Those properties motivate the hypothesis; they do not demonstrate that the baseline outperforms length or remains invariant across solver settings. A competitive implementation should reuse compiled problem structure, warm-start solver state, batch feature/model evaluation, and invoke child solves only after the proposal shortlist is stable.

B. Claim requirements for the complete evaluation

The headline comparison must pair PathFinder, three seeded simulated-annealing runs, length, dual-linear, proposal, and top-4 on identical kernel/architecture/II/budget cells. Full lookahead should be restricted to a frozen balanced subset because of cost. The parent ablation must use a genuinely no-parent checkpoint; a model trained or evaluated with parent dual features cannot be relabeled as no-parent. Exact quotienting must be evaluated with exhaustive equivalence checks on small cases and expansion reduction on larger cases. A3 remains held out until all models and thresholds are frozen.

C. Cost structure and implementation priorities

The method’s cost follows directly from the decision hierarchy. For a partial state, the residual program has placement variables proportional to the number of unmapped-operation/compatible-resource pairs and flow variables proportional to $|D_s||E_R|$. A proposal step requires one parent solve, graph and scalar feature construction, one batched GNN evaluation over the legal action group, and ordinary beam insertion. Top- k correction adds k child solves for each scored state; full lookahead adds $|\mathcal{A}(s)|$ child solves. Thus, the learned proposal is useful only when model and feature cost are small relative to the child solves it removes. The strict probe confirms that the present dominant term is the relaxation path, not neural inference, but the supplied timing does not isolate solver setup from numerical iterations.

Several optimizations preserve the method’s semantics. The sparse constraint matrices for a fixed DFG–MRRG pair can be compiled once, with partial-state occupancy, endpoint anchors, and optional knockout resources applied through parameters. Parent and child problems differ primarily in right-hand sides and residual capacities, making warm starts and factorization reuse natural. Action scoring can batch all targets and route choices for a state; graph embeddings can be reused across the action group; and state-independent architecture statistics can be cached by the architecture hash. These changes reduce repeated work without changing labels, the legal action universe, or the exact top- k definition.

The symmetry path has a different tradeoff. Canonicalization costs are proportional to the number of accepted joint transforms times the serialized state size. This overhead is justified only when it eliminates enough beam duplicates or relaxation solves. The implementation therefore needs per-instance accounting of accepted automorphisms, canonicalization time, merge count, expansions avoided, and downstream solves avoided. An exactness-preserving skip is preferable to a large group traversal that cannot amortize its own cost.

Finally, numerical diagnostics must remain visible. Near-zero or unstable duals can make the linear baseline uninformative even when the primal relaxation is feasible; an infeasible or unhealthy child solve cannot be converted into a finite oracle label. The artifact records solver status and residual checks so these cases can be separated from search failure. A complete evaluation should report the frequency of scoreable parent

TABLE VI
STRICT NATIVE TRMM PROBE. A DASH DENOTES NO COMPLETE MAPPING; TIMEOUT ROWS ARE EXCLUDED FROM QUALITY COMPARISONS.

Architecture	Method	Status	Mapped ops	Route cost	Parent solves	Child solves	Wall time (s)
A0 4×4 HyCUBE	Length	Done	11/11	71	0	0	117.44
A0 4×4 HyCUBE	Dual-linear	Timeout	6/11	–	4	0	601.33
A0 4×4 HyCUBE	Proposal	Done	11/11	71	22	0	468.48
A0 4×4 HyCUBE	Top-4	Timeout	5/11	–	0	–	602.12
A2 memory-constrained HyCUBE	Length	Done	11/11	71	0	0	146.61
A2 memory-constrained HyCUBE	Dual-linear	Timeout	6/11	–	4	0	602.22
A2 memory-constrained HyCUBE	Proposal	Done	11/11	71	22	0	481.49
A2 memory-constrained HyCUBE	Top-4	Timeout	5/11	–	0	–	602.00

states, rejected labels, fallback solves, and cache hits alongside mapping outcomes.

D. Threats to validity and limitations

Construct validity. The fractional formulation relaxes discrete signal conflicts and path choices; $J(s)$ is guidance, not a legality certificate. Route cost is also only one mapping-quality dimension, so the complete study must report success, minimum II, routing failures, and compiler time.

Internal validity. Solver tolerances, multiplier nonuniqueness, cache reuse, candidate truncation, prefix depth, and stochastic baselines can confound a score comparison. The protocol therefore records residual health, hashes cache contracts, holds the action universe fixed across deterministic variants, uses zero-prefix headline comparisons, and reports multiple simulated-annealing seeds with uncertainty. Reference mappings are isolated from search to avoid witness leakage.

External validity. The learned correction depends on the representativeness of sampled partial states, actions, kernels, and architectures. The frozen A3 holdout and compiler-derived kernels test only a limited form of transfer; conclusions should not be generalized to arbitrary CGRA organizations without additional native contracts.

Symmetry and current evidence. DFG graph automorphisms that do not preserve the complete total operation order are excluded, even if this leaves only the identity. Architecture transforms are admitted only under the ordered transition and score-equivariance contract. Exact groups may still be too large to enumerate, in which case the method reports a skip. The current artifact contains no completed quotient-versus-nonquotient terminal-set experiment and no evidence that relaxation guidance improves mapping quality.

IX. RELATED WORK

CGRA mapping. DRESC established retargetable modulo mapping with architecture-aware placement and routing [1]. CGRA-ME provides architecture modeling and both heuristic and optimization-based mapping infrastructure [2], [7]. Modulo graph embedding and hierarchical abstraction improve quality or scalability through structured search [3], [4]. Morpher integrates architecture description, compilation, simulation, and generation [10]. Rewire instead coordinates placement and routing for multiple DFG nodes by sharing routing

information during candidate construction [16]. A common-contract comparison remains necessary because that multi-node search space and termination policy differ from the atomic beam interface used here.

Learned mapping guidance. Deep reinforcement learning has been used for CGRA DFG mapping [9]; Zhuang et al. combine a GNN representation with reinforcement learning and routing exploration [15]; and LISA learns portable DFG labels that guide simulated annealing [8]. These works make GNN use, routing awareness, and learned CGRA guidance prior art. The narrower proposed distinction is the action-level decomposition into a residual-relaxation capacity baseline, a learned finite-action correction labeled by exact child relaxations, and optional exact top- k reevaluation under an unchanged native atomic-action contract. The current results do not establish superiority over these systems.

Learning expensive search decisions. Gasse et al. train a GNN to imitate strong branching, replacing repeated expensive child-relaxation evaluations with a learned branching policy [17]. The methodological analogy is direct: both learn from a costly relaxation-based expert and must evaluate prediction quality jointly with policy overhead. QUOTIENTFLOW differs in the CGRA-native placement-route action contract, the explicit parent-dual residual target, and optional exact top- k correction rather than pure imitation.

Congestion, duals, and symmetry. PathFinder updates congestion costs through negotiated routing [5], while convex duality provides local marginal sensitivity under regularity assumptions [11]. Symmetry reduction in integer and constraint programming relies on preserving the actual search problem, not merely an unannotated graph automorphism [18], [19]. Accordingly, QUOTIENTFLOW restricts quotienting to the complete ordered transition system and treats exhaustive action-bijection and terminal-set tests as mandatory evidence.

X. CONCLUSION

This paper specified QUOTIENTFLOW, a deterministic CGRA mapping framework that combines a residual-relaxation capacity baseline, learned action-level correction, optional exact top- k lookahead, and order-safe state-orbit reduction. The formulation separates local optimization sensitivity from the finite effect of a native atomic action, while the Morpher integration preserves a common legal action universe, isolates

reference mappings from search, and distinguishes mapping failure from operational failure. The supplied evidence supports narrow integration correctness and a two-architecture suffix probe; it also shows that repeated relaxation is currently too expensive. Acceptance-level claims require zero-prefix paired end-to-end results, learned-ranking and dual-stability measurements, conventional and closest learned baselines, exhaustive quotient equivalence tests, held-out transfer, and quality-versus-time analysis.

REFERENCES

- [1] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, “DRESC: A retargetable compiler for coarse-grained reconfigurable architectures,” in *Proc. IEEE Int. Conf. Field-Programmable Technology (FPT)*, 2002, pp. 166–173.
- [2] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. H. Anderson, “CGRA-ME: A unified framework for CGRA modelling and exploration,” in *Proc. IEEE Int. Conf. Application-Specific Systems, Architectures and Processors (ASAP)*, 2017, pp. 184–189.
- [3] H. Park, K. Fan, M. Kudlur, and S. Mahlke, “Modulo graph embedding: Mapping applications onto coarse-grained reconfigurable architectures,” in *Proc. Int. Conf. Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, 2006, pp. 136–146.
- [4] D. Wijerathne, Z. Li, A. Pathania, T. Mitra, and L. Thiele, “HiMap: Fast and scalable high-quality mapping on CGRA via hierarchical abstraction,” in *Proc. Design, Automation and Test in Europe Conf. (DATE)*, 2021, pp. 1192–1197.
- [5] L. McMurchie and C. Ebeling, “PathFinder: A negotiation-based performance-driven router for FPGAs,” in *Proc. ACM Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 1995, pp. 111–117.
- [6] M. Karunaratne, A. Kulkarni Mohite, T. Mitra, and L.-S. Peh, “HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect,” in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2017, pp. 45:1–45:6, doi: 10.1145/3061639.3062262.
- [7] S. A. Chin and J. H. Anderson, “An architecture-agnostic integer linear programming approach to CGRA mapping,” in *Proc. ACM/ESDA/IEEE Design Automation Conf. (DAC)*, 2018, pp. 1–6.
- [8] Z. Li, D. Wu, D. Wijerathne, and T. Mitra, “LISA: Graph neural network based portable mapping on spatial accelerators,” in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2022, pp. 444–459.
- [9] D. Liu, S. Yin, G. Luo, J. Shang, L. Liu, S. Wei, Y. Feng, and S. Zhou, “Data-flow graph mapping optimization for CGRA with deep reinforcement learning,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 12, pp. 2271–2283, 2019.
- [10] D. Wijerathne, Z. Li, and T. Mitra, “Accelerating edge AI with Morpher: An integrated design, compilation and simulation framework for CGRAs,” arXiv:2309.06127, 2023.
- [11] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [12] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *J. Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [13] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.
- [14] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems*, 2017, pp. 1025–1035.
- [15] Y. Zhuang, Z. Zhang, and D. Liu, “Towards high-quality CGRA mapping with graph neural networks and reinforcement learning,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2022, pp. 61:1–61:9, doi: 10.1145/3508352.3549458.
- [16] Z. Li, D. Wu, D. Wijerathne, D. Chen, H. Li, C. Tan, and T. Mitra, “Rewire: Advancing CGRA mapping through a consolidated routing paradigm,” in *Proc. ACM/IEEE Design Automation Conf. (DAC)*, 2025, pp. 1–7, doi: 10.1109/DAC63849.2025.11133240.
- [17] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi, “Exact combinatorial optimization with graph convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 15554–15566.
- [18] F. Margot, “Symmetry in integer linear programming,” in *50 Years of Integer Programming 1958–2008*, M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, and L. A. Wolsey, Eds. Berlin, Germany: Springer, 2010, pp. 647–686.
- [19] I. P. Gent, K. E. Petrie, and J.-F. Puget, “Symmetry in constraint programming,” in *Handbook of Constraint Programming*, F. Rossi, P. van Beek, and T. Walsh, Eds. Amsterdam, The Netherlands: Elsevier, 2006, pp. 329–376.

APPENDIX A AI USE

OpenAI ChatGPT was used for editorial restructuring, LaTeX assistance, and generation of the two conceptual schematic figures in this revision. It was not used to generate experimental measurements. The authors remain responsible for verifying all technical claims, equations, citations, source changes, and reported results before submission.